Pandas – consulta rápida

1 · Fundamentos

01 COMEÇAR

cap. 23 · §1–3

`import pandas as pd`	Convenção: sempre como pd.
`s = pd.Series([10, 20], index=["a", "b"])`	Coluna com etiquetas (índice).
`df = pd.DataFrame({ "nome": ["Ana", "Rui"], "idade": [23, 31]})`	Tabela a partir de um dicionário: cada chave é uma coluna.
`pd.DataFrame([ {"nome": "Ana", "idade": 23}, {"nome": "Rui", "idade": 31}])`	A partir de uma lista de dicionários: cada um é uma linha.

02 LER E GUARDAR

cap. 23 · §9.1, §12

`df = pd.read_csv("dados.csv")`	Ler um CSV (`read_csv`, com _).
`pd.read_csv("pt.csv", sep=";", decimal=",")`	CSV PT: separador `;` e decimal `,`
`pd.read_excel("dados.xlsx", sheet_name="Folha1")`	Excel; requer `openpyxl`.
`df.to_csv("saida.csv", index=False)`	Guardar sem a coluna do índice.

03 INSPECIONAR

cap. 23 · §4, §10.1

`df.head()` `df.tail(3)`	Primeiras 5 linhas · últimas 3.
`df.shape`	(linhas, colunas), sem `()`.
`df.columns` `df.index` `df.dtypes`	Colunas · índice · tipos.
`df.info()`	Tipos, não nulos e memória.
`df.describe()`	Resumo estatístico numérico.
`df["cidade"].value_counts()`	Quantas vezes aparece cada valor.
`df["cidade"].nunique()`	Número de valores distintos.

04 SELECIONAR

cap. 23 · §5

`df["nome"]`	Uma coluna → **Series**.
`df[["nome", "idade"]]`	Várias colunas → **DataFrame**. Lista dentro de `[]`.
`df.loc[0:2, "nome"]`	Por **etiqueta**. O fim **inclui**: linhas 0, 1 e 2.
`df.iloc[0:2, 0]`	Por **posição**. O fim **exclui**: linhas 0 e 1.
`df.loc[:, "idade":"vendas"]`	Todas as linhas, colunas de `idade` a `vendas`.
`df.at[0, "nome"]`	Um único valor (linha 0, coluna nome).

05 FILTRAR LINHAS

cap. 23 · §6

`df[df["idade"] > 25]`	Linhas que cumprem a condição.
`df[(df["idade"] > 25) & (df["cidade"] == "Porto")]`	`&` e · `|` ou · `~` não. Parênteses obrigatórios.
`df[df["cidade"].isin(["Porto", "Faro"])]`	Valor pertence a uma lista.
`df[df["idade"].between(20, 30)]`	Intervalo; inclui os dois limites.
`df[df["nome"].str.contains("an", case=False)]`	Texto contém, ignorando maiúsculas.
`df.query("idade > 25 and cidade == 'Porto'")`	O mesmo filtro escrito como texto; aqui usa-se `and`.

Tabela de exemplo usada nesta folha (`df`)

	nome	idade	cidade	vendas
0	Ana	23	Porto	120.0
1	Rui	31	Lisboa	NaN
2	Eva	27	Porto	80.0
3	Luís	40	Faro	200.0

Pandas – consulta rápida 2 · Transformar e resumir

06 CRIAR E ALTERAR

cap. 23 · §7

`df["total"] = df["preco"] * df["qtd"]`	Nova coluna, calculada para todas as linhas de uma vez.
`df.loc[df["idade"] > 30, "grupo"] = "sénior"`	Alterar só as linhas filtradas. As restantes ficam `NaN`.
`df = df.assign(dobro=df["vendas"] * 2)`	Nova coluna devolvendo outra tabela (encadeável).
`df = df.rename(columns={"nome": "cliente"})`	Mudar o nome de colunas.
`df = df.drop(columns=["dobro"])`	Remover colunas.

07 ORDENAR E ÍNDICE

cap. 23 · §8

`df.sort_values("idade", ascending=False)`	Ordem decrescente.
`df.sort_values(["cidade", "idade"])`	Várias chaves: primeiro cidade, depois idade.
`df.nlargest(3, "vendas")`	As 3 linhas com mais vendas (`nsmallest`: menos).
`df.set_index("nome")  df.reset_index()`	Usar uma coluna como índice · voltar a 0, 1, 2…

08 AUSENTES E DUPLICADOS

cap. 23 · §11

`df.isna().sum()`	Número de valores ausentes por coluna.
`df.dropna(subset=["vendas"])`	Remover as linhas sem vendas.
`df["vendas"].fillna(0)`	Substituir os ausentes por um valor.
`df.duplicated().sum()` `df.drop_duplicates()`	Contar · remover linhas repetidas.

09 TIPOS E TEXTO

cap. 23 · §9

`df["idade"].astype(int)`	Converter o tipo de uma coluna.
`pd.to_numeric(df["valor"], errors="coerce")`	Texto → número; o que não converte fica `NaN`.
`df["nome"].str.strip().str.title()`	Tirar espaços e pôr inicial maiúscula.
`df["nome"].str.lower()  .str.upper()`	Minúsculas · maiúsculas.
`df["codigo"].str.split("-", expand=True)`	Partir o texto em várias colunas.

10 DATAS

N2_01 · §10.1

`df["data"] = pd.to_datetime(df["data"], dayfirst=True)`	Converter texto `dd/mm/aaaa` em data.
`df["data"].dt.year  .dt.month  .dt.day_name()`	Extrair ano, mês ou dia da semana.
`df.set_index("data").resample("ME")["vendas"].sum()`	Total por mês (`"ME"` = fim do mês).

11 ESTATÍSTICAS E GRUPOS

cap. 23 · §10 · N2_01 · §10

`df["vendas"].mean()  .median()  .sum()  .min()  .max()  .std()`	Medidas de uma coluna. Ignoram `NaN`.
`df.groupby("cidade")["vendas"].sum()`	Total de vendas por cidade.

`df.groupby("cidade").agg(`
`  total=("vendas", "sum"),`
`  clientes=("nome", "count"))`
	Várias medidas com nomes escolhidos por ti.

`df.groupby("cidade")["vendas"].transform("mean")`	Média do grupo em cada linha: mantém o número de linhas.
`pd.cut(df["idade"], bins=[0, 30, 100], labels=["jovem", "sénior"])`	Agrupar números em classes.

Pandas – consulta rápida 3 · Combinar, exportar e armadilhas

12 JUNTAR TABELAS

N2_01 · §9

`pd.merge(vendas, clientes, on="id")`	Por omissão: só chaves comuns (inner).
`pd.merge(vendas, clientes, on="id", how="left")`	Toda a esquerda; NaN sem par.
`… how="outer"`	Todas as chaves dos dois lados.
`… left_on="id_cliente", right_on="id"`	Chave com nomes diferentes.
`… validate="many_to_one"`	Erro se a direita repetir chaves.
`… indicator=True`	Coluna _merge com a origem.

13 EMPILHAR E REMODELAR

N2_01 · §9.1, §10.2

`pd.concat([jan, fev], ignore_index=True)`	Empilhar tabelas com as mesmas colunas.
`df.pivot_table(index="cidade", columns="mes", values="vendas", aggfunc="sum", fill_value=0)`	Tabela dinâmica: cidades × meses.
`df.melt(id_vars="nome", var_name="mes", value_name="vendas")`	Formato largo → longo.
`pd.crosstab(df["cidade"], df["grupo"])`	Contagens cruzadas de duas colunas.

14 EXPORTAR

N2_01 · §11.1 · N2_02 · §2

`df.to_excel("saida.xlsx", index=False)`	Excel (requer openpyxl).
`df.to_json("saida.json", orient="records", force_ascii=False, date_format="iso")`	JSON com acentos e datas legíveis.
`df.to_parquet("saida.parquet")`	Guarda os tipos; requer pyarrow.

! ARMADILHAS FREQUENTES

× errado · ✓ certo

Nome da função. É `read_csv`, não um atributo `read`.

✗ ~~`pd.read.csv("dados.csv")`~~

✓ `pd.read_csv("dados.csv")`

Combinar condições. `and`/`or` não funcionam com colunas inteiras.

✗ ~~`df[df["idade"] > 25 and df["cidade"] == "Porto"]`~~

✓ `df[(df["idade"] > 25) & (df["cidade"] == "Porto")]`

Atribuição encadeada. No pandas 3 não altera a tabela original.

✗ ~~`df["vendas"][df["idade"] > 30] = 0`~~

✓ `df.loc[df["idade"] > 30, "vendas"] = 0`

Esquecer de guardar. O método devolve uma tabela nova.

✗ ~~`df.dropna()`~~

✓ `df = df.dropna()`

Várias colunas. Os nomes vão dentro de uma lista.

✗ ~~`df["nome", "idade"]`~~

✓ `df[["nome", "idade"]]`

Procurar ausentes. NaN nunca é igual a nada, nem a si próprio.

✗ ~~`df[df["vendas"] == np.nan]`~~

✓ `df[df["vendas"].isna()]`

Médias com colunas de texto. Dá `TypeError`.

✗ ~~`df.mean()`~~

✓ `df.mean(numeric_only=True)`

Atributo, não método. `shape`, `columns` e `dtypes` não levam `()`.

✗ ~~`df.shape()`~~

✓ `df.shape`

Fluxo de trabalho típico

1 Ler	2 Inspecionar	3 Limpar	4 Transformar	5 Agregar	6 Exportar
`read_csv`	`info`	`dropna`	`assign`	`groupby`	`to_csv`